

Contents

1 draft: text capture	1
1.1 outline	1
1.2 draft	1
1.2.1 what is web scraping?	1
1.2.2 TODO here: permission based model to create "fair exchange"	8

1 draft: text capture

1.1 outline

- web scraping
 - what is web scraping?
 - ☒ html, bytes
 - ☐ network, TCP/IP ??
 - ☐ blockers
 - blocking increases due to AI scraping.
 - workarounds: changing useragent, using ghost browser.
 - ☐ theorizing standards ??
 - scraping congress.gov
 - *
- conclusion
 - eve sedgwick on reading practices?
 - Bowers vs Hardwick, Bostock v. Clayton County.

1.2 draft

1.2.1 what is web scraping?

1. from bytes to HTML

Web scraping is the extraction of information from websites. It most often involves (with some exceptions, explained below) working with data in HTML format, with elements and attributes.

```

<!DOCTYPE html>
<html>
<body>

<h1 class="header">First level heading element</h1>

<p>A paragraph element with a <a href="/">link</a> inside of it</p>

</body>
</html>

```

Websites are stored in web servers, accessed over a network. Web data travels over the network in the form of bytes, the smallest digitally addressable data unit. When the bytes reach the web browser, they are re-encoded into HTML language, which is assembled into a web page readable by human eyes.

2. HTTP

Web *scraping* is possible by manipulating HTTP, or HyperText Transfer Protocol, which is the language of the network. A web scraper uses code to manipulate an HTTP request so that it receives the website data in the form of code.

Web scraping is, at its core, about mimicking a typical browser request. Using HTTP protocols, it makes a request to a web server much like a user makes a request by typing in a URL into the browser. The data it gets back from the HTTP request is also the same as the data that comes to a browser after searching a URL—only the data is intercepted before in the form of bytes, rather than assembled into a web page visible through a browser.

```
r = requests.get("https://www.nytimes.com/")
```

You can scrape a website with only five lines of code. Below is an example that scraps the home page of *The New York Times* website using the `bs4` and `requests` libraries in python.

```

from bs4 import BeautifulSoup
import requests

```

```

r = requests.get("https://www.nytimes.com/")
source = r.content
soup = BeautifulSoup(source, 'html.parser')

```

The `requests` library is one of the most popular libraries in the python programming language. Enabling programs to send and receive data through HTTP networks (i.e., the internet), it is the bedrock for virtually all web scraping and API tasks. The `get()` function from `requests` can access the *NYTimes* homepage, which is saved in the form of bytes via the `content` property. Then, using `bs4`, a python library for working with HTML data, the data is parsed into an object called `soup` (following `bs4`'s naming conventions), which is a pythonic object of HTML data.

The beauty of `bs4` is that it makes HTML—a notoriously fussy language that is the reason why so many web frameworks exist in the first place, attempting to abstract the user from working directly in HTML—accessible through simple python syntax.

For example, the expression `soup.title` returns the following output:

```

>>> soup.title
<title data-rh="true">The New York Times - Breaking News, US News, World News and

```

Similarly, other HTML elements can now be accessed using python dot syntax.

```

>>> soup.p
<p>Make sense of the day's news and ideas.</p>

>>> soup.a
<a class="css-kgn7zc" href="#site-content">Skip to content</a>

>>> soup.h1
<h1 class="css-1dv1kvn">New York Times - Top Stories</h1>

```

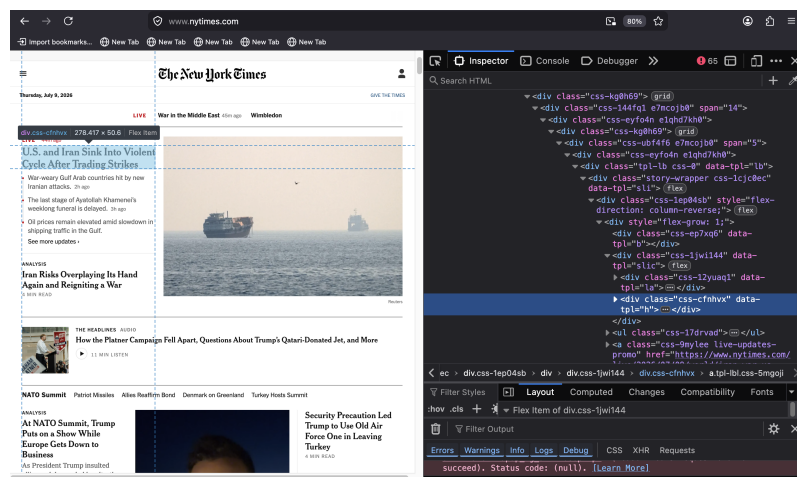
Further properties like `text` (for just text) and functions like `find()` are also possible. Functions can be further specified when combined with the `class_` parameter, to search the HTML with more precision. Due to python syntax, they can also be chained together to create complex expressions. These building blocks enable programmers to navigate the HTML tree with granularity.

```
>>> soup.h1.text
'New York Times - Top Stories'

>>> soup.find('div', class_ = 'css-cfnhvx')
<div class="css-cfnhvx" data-tpl="h"><a class="tpl-lbl css-5mgoji" data-tpl="l" href="https://www.nytimes.com/2020/07/09/middleeast/iran-sink-into-violent-cycle-after-trading-strikes.html">U.S. and Iran Sink Into Violent Cycle After Trading Strikes'

>>> soup.find('div', class_ = 'css-cfnhvx').text
'U.S. and Iran Sink Into Violent Cycle After Trading Strikes'
```

HTML classes for an element can be found by using built-in browser tools like the web inspector. On the webpage, right click on the relevant text and select **inspect** from the dropdown menu. The element will be highlighted with information for the class attribute.¹



Using `requests`, `bs4`, and the web inspector, one can easily grab a list of the current headlines from the *NYTimes* homepage.

```
>>> for i in soup.find_all('div', class_ = 'css-cfnhvx'):
    print(i.text)
```

¹Note that across programming languages like HTML and python, terms like "attributes" mean different things and in practice function very differently. In HTML, an attribute is data contained within an HTML element, like a value specifying a CSS class, such as `<h1 class=title>New York Times</h1>`. By contrast, in python an attribute is information associated with an object model, like `r.content` or `r.status_code`, which is data associated with an HTTP request object, `r`.

U.S. and Iran Sink Into Violent Cycle After Trading Strikes
 Iran Risks Overplaying Its Hand Again and Reigniting a War
 At NATO Summit, Trump Puts on a Show While Europe Gets Down to Business
 Security Precaution Led Trump to Use Old Air Force One in Leaving Turkey
 ‘A Slow-Rolling Disaster’: Inside the Implosion of the Platner Campaign
 What’s Next for Democrats in the Senate Race in Maine
 How Democrats and Republicans Diverge on Sexual Assault Allegations
 Platner Suspends Senate Bid in Maine After Rape Accusation
 Trump’s Loyal Defender at the Vatican
 Trump Says He’ll Ask Supreme Court to Rehear Citizenship Case, an Unlikely Event
 Can Trump’s Arch Be So Tall? A Law May Be Redefined to Get to Yes.
 Morocco Faces France and Its Star, Mbappé, in Quarterfinals
 A Colonial Past Mixes With the Global Present
 How Morocco’s Diaspora Became the Team’s Driving Force
 13 Red Cards Have Been Given at the World Cup. Not All Have Been Equal.

Anything else to say about HTTP?

3. **TODO** Blockers

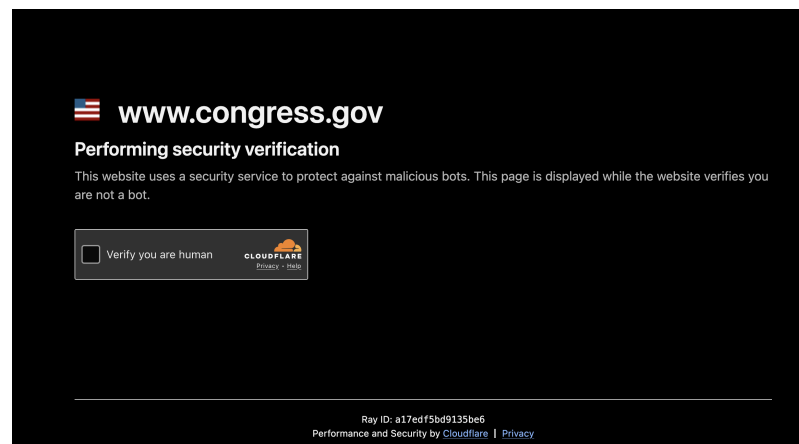
Web servers, unfortunately for the scraper, often block automated requests to their data. They use various forms of "blockers" or mechanisms that deny the HTTP request, sending back a code 403, or "Forbidden" message.

Each year of the past few years, bot traffic has increased exponentially, coming mostly from major tech and AI companies, especially OpenAI, META, Google, and Anthropic. According to Cloudflare, 2025 saw increases of 19% in bot traffic (See Fig #) (Cloudflare, "From Googlebot to GPTBot: who’s crawling your site in 2025"). It is no surprise that traffic has increased mostly from these companies, whose ML model training requires ever increasing amounts of data.

Rank	Bot Name	Share (May 2024)	Rank	Bot Name	Share (May 2025)
1	Bytespider	42%	1	GPTBot	30%
2	ClaudeBot	27%	2	ClaudeBot	21%
3	Amazonbot	21%	3	Meta-ExternalAgent	19%
4	GPTBot	5%	4	Amazonbot	11%
5	Applebot	4.1%	5	Bytespider	7.2%

As automated scrapers increase in frequency, so do the bot blockers. Over the past two years, Cloudflare, which runs on 20% of total internet pages and 50% of pages based in the US, has added increasing bot detection like "Challenges" and "Turnstiles" to its web pages. Some of these operate under the surface and virtually invisible to the user, such as checking the HTTP request metadata like the "User-Agent." By default, the User-Agent is set to the name of web browser and operating system making the request. For example, my Firefox browser running Mac OSX has the User-Agent `Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:152.0) Gecko/20100101 Firefox/152.0`. The python `requests` library, by contrast, sets the User-Agent to `python-requests/2.32.3`, making any HTTP request using that library easy for blockers to detect.

Beyond examining HTTP request data, bot blockers will use JavaScript code to check if the request is from a real web browser, rather than from an automated program. JavaScript, which is a programming language native to web browsers (and discussed in more detail in Chapter 4: Text Display), can inject challenges for the browser to solve, challenges which may or may not involve the user. All of these challenges might require some processing from the browser, and increasingly, they may require the user's interaction (to confirm that the request comes from a real browser, not a bot). One popular challenge is the "security verification" checkbox, which is increasingly visible across the internet (See Fig #).



While these kinds of blockers are more advanced, there are ways of

overcoming them. One effective method involves using a virtual or "ghost" browser through python libraries like `selenium`. Instead of sending an HTTP request via a python script, it routes the request through an actual web browser—though one that is operated remotely by the python script. When running, the code will open a browser window, enter a URL, and navigate a web page to click on links, all without the input of the user. In other words, it operates like a "ghost" on the computer. One major drawback to this method is that it requires more processing and a display (which many "headless" scraping servers do not have). The computational expense of this workaround detracts large-scale web crawling.

As a result of increased bot scraping, internet security companies like Cloudflare have recently moved to what they call a "permission-based model," where bots must pay for seamless access to certain domains. A blog post frames the new policy as a logical step in the mass data extraction environment.

For decades, the Internet has operated on a simple exchange: search engines index content and direct users back to original websites, generating traffic and ad revenue for websites of all sizes. This cycle rewards creators that produce quality content with money and a following, while helping users discover new and relevant information. That model is now broken. AI crawlers collect content like text, articles, and images to generate answers, without sending visitors to the original source – depriving content creators of revenue, and the satisfaction of knowing someone is viewing their content. (“Cloudflare Just Changed How AI Crawlers Scrape the Internet-at-Large; Permission-Based Approach Makes Way for A New Business Model”)

The argument bases itself on the traditional purpose of large-scale web crawling, which was to populate and otherwise assist search engines. This activity of taking content in the form of search results was legitimized in a famous copyright lawsuit against Google, *The Authors Guild Inc., et al. v. Google, Inc.* (2015). In this case, the judge ruled that search results, such as that from Google Books, were considered "fair use" because they point users to the original source without diminishing its value.

1.2.2 TODO here: permission based model to create "fair exchange"

Cloudflare points out that today's web scrapers and crawlers do not point users back to the original source; rather, the AI training process severs the data completely from its source, so that this connection cannot be monetized, "depriving content creators of revenue... and satisfaction." As a solution, Cloudflare offers the "permission based model," where companies

But there is a colonial/capitalist dynamic in the subtext here. Applying Couldry and Meija's lense of the "data grab," we can see

"rewards creators" - actually, rewards content owners; "helping users discover" - actually, extracting data from users

"For decades, the Internet has operated on a simple exchange: search engines index content and direct users back to original websites, generating traffic and ad revenue for websites of all sizes. This cycle rewards creators that produce quality content with money and a following, while helping users discover new and relevant information. That model is now broken. AI crawlers collect content like text, articles, and images to generate answers, without sending visitors to the original source – depriving content creators of revenue, and the satisfaction of knowing someone is viewing their content." ("Cloudflare Just Changed How AI Crawlers Scrape the Internet-at-Large; Permission-Based Approach Makes Way for A New Business Model")

Cloudflare has ramped up its blocking technology.

"In September 2024, Cloudflare introduced the option to block AI crawlers in a single click. More than one million customers have since chosen this option, meant to be an aggressive but easy solution that halts scraping while they determine their AI strategy." ("Cloudflare Just Changed How AI Crawlers Scrape the Internet-at-Large; Permission-Based Approach Makes Way for A New Business Model")

As a result, Cloudflare moves to a "permission based model" that can create a "fair exchange".

"“Cloudflare’s innovative approach to block AI crawlers is a game-changer for publishers and sets a new standard for how content is respected online. When AI companies can no longer take anything they want for free, it opens the door to sustainable innovation built on permission and partnership,” said Roger Lynch, CEO of Condé Nast. “This is a critical step toward creating a fair value exchange on the Internet that protects creators, supports quality journalism and holds AI companies accountable.”” (“Cloudflare Just Changed How AI Crawlers Scrape the Internet-at-Large; Permission-Based Approach Makes Way for A New Business Model”)